Code Jam 2019 - World Finals

Analysis: Board Meeting

Test set 1

In the first test set, there is only one king. There are various ways for us to deduce its position; we will describe one that requires three queries and is relatively easy to understand.

First, we query (-1000000, 1000000), which is the upper left corner of the board; let A_1 be the result. It tells us which J-shaped region of the board the king is in, illustrated as follows starting from the upper left corner:

```
0123...
1123
2223
3333
.   .
.     .
.       .
```

Next, if we did not happen to get lucky and find the king, we query (-1000000 + A_1, 1000000 - A_1). That is, we query the corner of that J-shaped region. Call the result A_2; it tells us how far away from the corner the king is, within that region. However, we do not know whether the king is to the left of or above the corner.

Finally, if we have still not found the king, we query (-1000000 + A_1 - A_2, 1000000 - A_1). That is, we guess that the king is to the left of the corner. If the result is 0, then we have found the king; otherwise, we know the king is above the corner, at (-1000000 + A_1, 1000000 - A_1 + A_2).

Test set 2

With more than one king, we find that it is not possible to determine the exact location of each king. For example, we cannot distinguish between a case with kings at (+1, 0) and (-1, 0) and a case with kings at (0, +1) and (0, -1). So it must be possible to answer queries with less complete information about the kings' locations.

The "L^∞" metric in the problem, for which we take the maximum of the absolute differences of two coordinates, is a little inconvenient to work with directly, which results in the fairly ad-hoc nature of the test set 1 solution above.

Converting to diagonal coordinates makes things simpler. If a point is at (x, y) in the original coordinates, let (u, v) = (x + y, x - y) / 2 be its diagonal coordinates. Then if the diagonal coordinates of two points are (u1, v1) and (u2, v2), the distance between them is |u1 - v1| + |u2 - v2|. So if the kings are located at diagonal coordinates (ui, vi) for i= 1, ..., N, then the result of a query for the point (u, v) is Σ_i (|u - ui| + |v - vi|) = (Σ_i |u - ui|) + (Σ_i |v - vi|)

We can handle these two sums separately. To compute the first one, we need to know which "downright-sloping" diagonal lines the kings lie on, and to compute the second one, we need to know which "upleft-sloping" diagonal lines the kings lie on.

Now we need a method to compute these diagonals. Consider the point (-2M, 0). This point is far enough to the left that, if we propose this point, the total distance we get will be the sum of the differences in x coordinates. If we propose (-2M, 1), we will get the same answer, unless there is a king at (-M, -M), in which case we will get an answer one higher.

More generally, for a positive integer K, let d(K) for a positive integer K be the difference in the results of proposals of (-2M, K) and (-2M, K-1). d(K) will be equal to the number of kings below the diagonal x + y = -2M + K. We can use binary searches to find the places where d(k) changes value, and the magnitudes of those changes, to find out which diagonals of the form x + y = c contain kings, and how many they contain.

Similarly, by proposing points (-2M, -K), we can find out how many kings lie on the diagonals that go in the other direction. Once we have this data, we are able to compute the sums above and respond to queries from the judge.

Code Jam 2019 - World Finals

Analysis: Go To Considered Helpful

There are two kinds of programs: those that terminate, and those that don't!

For any set of instructions for Marlin that does not loop indefinitely (because it reaches the end of the program), there is an equivalent program that has no goto instructions, and this is the shortest way to write that program.

Similarly, if a program loops indefinitely, there is an equivalent program that consists only of move instructions with a single goto at the end of the program, and this is the shortest way to write that program.

So we only need to consider those two cases: programs that consist only of move instructions, and programs with only a single goto, as the last instruction. Then we take the shortest of all of these.

We can easily find a minimal program of the first type with a breadth-first search (BFS) which starts at `M` and stops when it reaches `N`, avoiding `#`s. If the search terminates without reaching `N`, then we can output that the case is impossible.

Finding a minimal program of the second type is harder. The program can be split into two sections which compute two separate paths: an initial path P produced by the instructions which do not repeat, and then a path Q which repeats until `N` is reached.

Let B be the point where P ends. We can use a BFS, as in the previous case, to find a minimal P for all possible points B.

Optimizing Q is slightly more involved. The first time through Q, Marlin walks through some set of grid cells. The second time through, Marlin walks through grid cells of the same pattern, but offset by some displacement vector D. This continues for some number of iterations K until Marlin reaches the `N` cell. So the first iteration of Q takes Marlin from B to B+D, but the path must avoid not only cells with a `#`, but also cells which have an equivalent cell in a later iteration which is a `#`. Additionally, the first iteration of Q must include a cell which, in a later iteration, will be `N`.

It is not easy to satisfy these constraints unless we already know the displacement D, and the number of iterations, K. So, we loop over all possibilities for these.

Inside this loop, we determine the shortest program for all possibilities for B, for the given values of D and K. We split Q into two parts, Q_1 and Q_2. Q_1 contains the instructions which repeat K times, and reach cell `N` on the final iteration. Q_2 contains the instructions which only repeat K-1 times. (If we reach `N` at the end of an iteration, Q_2 is empty.)

Let N be the location of the cell containing `N`. Q_1 is a path from B to N-(K-1)×D, and Q_2 is a path from N-(K-1)×D to B+D.

The path for Q_1 can only touch cells X such that X+i×D is empty for 0<i<K. We do a BFS from N-(K-1)×D, using these cells, to find an optimal Q_1 for all possibilities for B.

The path for Q_2 can only touch cells X such that X+i×D is empty for 0<i<K-1. We do a BFS from N-(K-1)×D, using these cells, to find an optimal Q_2 for all possibilties for B.

Finally, we loop over all possible cells for B, and sum up the lengths of the shortest P, Q_1 and Q_2, plus one for the goto statement.

Running time

Let max(**R**,**C**)=N. For the outer loop that iterates over D and K, there are $O(N^2)$ choices for D and O(N) choices for K. However, not all combinations are possible. If the number of iterations, K, is large, then the displacement D for each iteration must be small; otherwise Marlin's path would have to leave the grid to complete K iterations. The total number of valid combinations of D and K is $O(N^2)$ — we leave the proof as an exercise for the reader.

Inside the loop, setting up the grids for the BFS, running the two BFSs, and trying all values for B all take $O(N^2)$ time. So the loop takes $O(N^4)$ time.

The BFS outside the loop that computes all optimal paths P takes $O(N^2)$ time, and so does the search for the optimal solution with no goto statements. So the overall solution is $O(N^4)$, and runs fast enough to solve both test sets.

Being less careful can result in an $O(N^5)$ solution — for example, by doing O(K) work to check whether a cell is valid for the two BFSs inside the loop. This is sufficient to solve test set 1.

An exponential-time solution that naively tries all possibilities for Q is unlikely to work even for test set 1, since the maximum length of Q is too large.

Code Jam 2019 - World Finals

Analysis: Juggle Struggle: Part 1

Test Set 1 (Visible)

We are given a set of 2**N** points that we are to pair into **N** line segments such that they all intersect each other. That makes the set of line segments magnificent.

We can extend any line segment S into a full line L that divides the plane in two. In a magnificent set of line segments, since S intersects all other segments, all those other segments have one endpoint on each side of L (no other point can be on L or it would be collinear with the endpoints of S). This means that if we pick a point P in the input, it must be paired with another point Q such that the line that goes through both leaves exactly the same number of other points on each side. This hints at an algorithm: choose a point P, find a Q according to the definition above, pair P and Q, and repeat. However, what if there is more than one such Q?

One way to deal with the problem is by choosing P intelligently. For example, if we choose a leftmost point as P, all candidates for Q end up in the right half of the plane cut by a vertical line through P (with at most one other point possibly on the line). Consider a line rotating clockwise centered on P, going from vertical to vertical again (rotating 180 degrees). At the starting point, there are no points on the left side of the line, because P is leftmost. As it rotates, it will pass over all points, repeatedly adding one point to one side and removing one from the other. Since there are no collinear triples of points, this shift is always by 1. This means there is exactly one such line that has P and one more point on it, with **N**-1 points on each side. Moreover, if we sort all non-P points by the angle they form with P (choosing 0 to coincide with the vertical line), the point that ends up on the line is exactly the median of the sorted list.

Choosing P as a leftmost point gives a unique choice of another point Q to pair P with. We can then remove both and continue. The algorithm requires **N** iterations, and in each of them we must find a leftmost point, and sort O(**N**) points by angle. Calculating the cosine of the angles to compare without loss precision takes constant time. Picking the median and removing the pair of points all can be done in O(**N**) time or better. Therefore, the overall algorithm requires O($\mathbf{N}^2$ log **N**) time, and it is enough to solve Test Set 1. If one uses a linear algorithm to find the median instead of sorting, it would improve the time complexity to O($\mathbf{N}^2$). However, sorting to find the median is likely to be faster in practice due to constant factors.

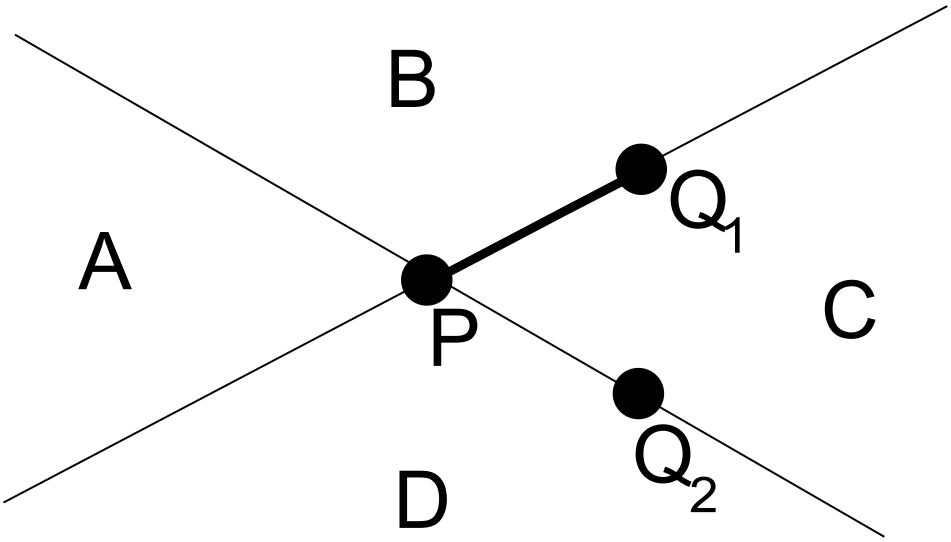
A consequence of the reasoning above is that the solution is actually unique. Another observation is that any point in the convex hull of the points has the same property as a leftmost point, because the definition is invariant through rotations and any point in the convex hull can be made a leftmost point through a rotation. But a leftmost is one of the easiest to find.

Test Set 2 (Hidden)

There is another approach requiring an additional proof but leading to a slightly simpler algorithm for Test Set 1. Most importantly, it leads us one step closer to solving Test Set 2.

The additional key observation for this approach is that if a set of 2**N** points admits a magnificent pairing, then for *every* point P in the set (not just those in the convex hull) there is exactly one Q such that the line through P and Q leaves half the points on each side. The fact that there is at least one is immediate from the existence of a magnificent arrangement. The argument that there cannot be more than one requires more thought.

Assume there is a point P and two other points Q_1 and Q_2 such that both of the lines that go through P and each Q_i have half of all the other points on each side. Moreover, without loss of generality, assume Q_1 is the one that actually pairs with P in the magnificent pairing (we know it is unique). The following picture illustrates the situation.



In the picture, we have labeled the four areas into which the two lines divide the plane. Let A, B, C and D be the sets of input points contained in each area (not including P, Q_1 or Q_2). Notice that any point in A must be paired with a point in C in order for the produced segment to intersect PQ_1. Similarly, any point in D must be paired with a point in B. All points below the line that goes through P and Q_2 are in either A or D, which means there are $|A| + |D|$ of them. The number of points above, on the other hand, is $|B| + |C| + 1$. Since we showed $|A| \leq |C|$ and $|B| \leq |D|$, $|B| + |C| + 1 \geq |A| + |D| + 1 > |A| + |D|$. This contradicts the assumption that the line that goes through P and Q_2 has the same number of input points on each side.

This observation by itself only allows us to avoid the "find a leftmost point" step of the previous algorithm, which does not change its overall time complexity. The definite improvement is to more quickly shrink the set of points that we must consider, so that all steps within the iteration take less time. To do that, consider that after we have paired M pairs of points, the lines through each pair divide the plane into 2^M areas, with only the outside areas (the ones with unbounded area) possibly containing leftover points. Moreover, each point is to be paired with one in the area that is opposite in order to cross all the lines, which is a necessary condition to intersect all line segments. When we create a new pair, exactly two of the areas are split in half. The idea is then: instead of removing the new pair and continuing with the full set, continue recursively with two separate sets. If we consider sets X and Y coming from opposite areas, split after pairing into X_1 and X_2 and Y_1 and Y_2, respectively, then we need to solve recursively for X_1 and Y_1 separately from solving for X_2 and Y_2 (assuming the areas are labeled such that X_1, X_2, Y_1, Y_2 appear in that order when going through them in clockwise order).

The last thing to do is to make sure we split X and Y more or less evenly with the new line. Notice that if we restrict ourselves to start with a point P that is part of the convex hull of X or Y or $X \cup Y$, this might be impossible (i.e., all those pairs may split X and Y, leaving most remaining points on one side). Hence the need for the property with which we started this section. However, notice that if we sort all pairs to be made between points in X and points in Y by the slope of their produced segments, the first and last will split X and Y into an empty set and a set with $|X| - 1$ points (notice that $|X| = |Y|$). The second and next-to-last will split them into a set with 1 point and a set with $|X| - 2$ points. The i-th will split them into a set with $i - 1$ points and a set with $|X| - i$ points. Therefore, if we choose randomly, we end up with a recursion similar to quicksort's, in which the reduction on the size of the sets is expected to be close enough to always partitioning evenly. Since the time it takes for the non-recursive part of a set of size M is

O(M log M) — notice that calculating the partitions is only an additional linear time step — the overall expected time complexity of this recursive algorithm is ($\mathbf{N}$ $\log^2$ $\mathbf{N}$).

Code Jam 2019 - World Finals

Analysis: Juggle Struggle: Part 2

We can represent each pair of jugglers from the input as the endpoints of a segment. The problem then asks us to find two of those segments that do not intersect, or report that there is no such pair.

Test Set 1 (Visible)

Test Set 1 can be solved by checking every possible pair of segments to see if they intersect. Let pq be the segment between points p and q, and rs be the segment between points r and s; then these segments intersect if and only if

- (r - p) × (q - p) has the same sign as (s - p) × (q - p), and
- (p - r) × (s - r) has the same sign as (q - r) × (s - r),

where × stands for the cross product. This works only when no three of the points are collinear, but that is the case in this problem. Since each check takes constant time, this algorithm runs in $O(\mathbf{N}^2)$ time.

Test Set 2 (Hidden)

Let L_i be the line that fully contains segment S_i. Let S_i and S_j be two segments that do not intersect. If L_i and L_j are not parallel, at least one of S_i and S_j does not contain the intersection between L_i and L_j. We can find for every input segment S_i, we can find whether its line L_i intersects some other line at a point outside of S_i (or not at all). Let F be the set of all such segments. Then, every non-intersecting pair contains at least one segment in F and the size of F is at most 25. We can do a linear pass over all other segments to see whether or not they intersect each segment in F to find the segments not in F that also participate in non-intersecting pairs. We focus now on finding all segments to put in F.

We now present three algorithms. The first two are ultimately similar: the first one uses less advanced theory, but requires more problem-specific proofs because of it. They both require somewhat large integers. For C++, `__int128` is enough to represent all values, but since we need to compare fractions that are the ratio of two `__int128`s, we'll need special comparison code. For Java, `BigInteger` will do. The third algorithm is a separate approach that uses a more advanced data structure, and requires only 64-bit integer arithmetic.

A solution using less advanced previous knowledge

First, assume there is a purely vertical segment (that is, its two endpoints have the same x coordinate). If we find more than one of those, we add all of them to F, since they don't overlap. If we find a single one, we can check it against all others like in the previous solution in linear time. In what follows, we assume no vertical segment is present.

Consider the extension of each segment S_i to a full line L_i. We will find the x coordinates of the leftmost and rightmost intersection of L_i with all other L_js and check that they are inside the range of x coordinates where S_i exists. If one of those intersections is not inside that range, then we found one or two segments to put in F. Notice that finding all rightmost intersections is equivalent to finding all leftmost intersections in the input symmetric to the y axis, so if we have an algorithm that finds the leftmost ones, we can just run it twice (and reflecting the input takes

linear time). Moreover, suppose we restrict ourselves to the leftmost intersection of L_i such that L_i is above the other intersecting line to the left of the found intersection. Let us call these "leftmost above intersections". We can use an algorithm that finds those intersections once on the unchanged input and once on the input reflected across the x axis to find the analogous "leftmost below intersections". In summary, we develop an algorithm that finds "leftmost above intersections" and then run it 4 times (using all combinations of reflecting / not reflecting the input across each axis), to find all combinations of "leftmost/rightmost below/above intersections".

To find all "leftmost above intersections", the key observation is that if two lines L_1 and L_2 intersect at x coordinate X, and L_2 is below to the left of the intersection, then L_2 cannot participate in any leftmost below intersection at coordinates X' > X. L_2's own intersections at coordinates X' > X are not leftmost. If L_2 intersects an L_3 that is below L_2 to the left of their intersection at X' > X, then L_3 intersects L_1 to the left of X' because of continuity: L_1 is below L_2 to the right of X.

This leads to the following algorithm: let X_0 be the minimum x coordinate among all endpoints. Sort the lines by y coordinate at X_0. Let L_i be the line with the i-th highest y coordinate. We iterate over the lines in that order, while keeping a list or ranges of x coordinates and which previously seen line is below all others there, since that is the only one that can produce leftmost below intersections in that range. We keep that list as a stack.

At the beginning, we push (X_1, 1) onto the stack, where X_1 is the maximum x coordinate among all input points. This signifies that L_1 is currently below in the entire range of x coordinates. Then, we iterate through L_2, L_3, ... $L_{\mathbf{N}}$. When processing L_i, we find the x coordinate of its intersection with L_j and call it X, where (j, X') is the top of the stack. We check the intersection to see if it is within the x coordinate range of the two corresponding segments. Then, if X < X', we simply push (i, X) onto the stack. Otherwise, we pop (j, X') from the stack and repeat, since j was not the line below all others at X. Notice that this keeps the stack sorted increasingly by line index and decreasingly by intersection coordinate at all times.

Since every line is processed, pushed onto the stack and popped from the stack at most once, and everything else can be done in constant time, this iteration takes linear time. Other than that, the sorting by y coordinate takes time O(**N** log **N**), which is also the overall time complexity of the entire algorithm, since it dominates all other linear steps.

Notice that the way we use the stack in the above algorithm is quite similar to how a stack is used in the most widely known algorithm to compute the convex hull of a set of points. As we show in the next section, that is no coincidence.

Using point-line duality to shortcut to the solution

In this solution we change how we find leftmost intersections. Treating vertical lines and reflecting to find rightmost intersections, and the way to use leftmost/rightmost intersections to find the solution to the problem, are the same as in the solution above.

To find the leftmost intersections, we can apply the point-line duality to the input. With duality between points and lines, a line y=mx+b in the original space can be represented as the point (m, -b) in the dual space. Similarly, a point (a, b) in the original space can be represented as a line of the form y=ax-b in the dual space. Notice that the dual space of the dual space is the original space. Vertical lines have no corresponding point. This duality has the property that when two lines L_1 and L_2 intersect in the original, their intersection point P corresponds to the line dual(P) in the dual space goes through the points dual(L_1) and dual(L_2).

Thus, if we take all lines that are extensions of input segments and consider the points that correspond to them in the dual space, the leftmost intersection for a given line L_1 occurs when intersecting L_2 such that the slope of the segment between dual(L_1) and dual(L_2) is minimal.

We now work on the dual space with an input set of points, and for each point P we want to find another point Q such that the slope of PQ is minimal. For each point in the convex hull of the set, the minimal slope occurs between that point and the next point in the convex hull. For points not in the convex hull, however, the appropriate choice is the temporary "next" point of the convex hull as calculated by the Graham scan. This leads to similar code as for the solution above, but using the duality saves us quite a few hand-made proofs. Just as for the algorithm above, all of the steps of this algorithm take linear time, with the exception of the sorting step needed for the Graham scan, yielding an overall O(**N** log **N**) algorithm.

A solution using incremental convex hull

Another solution requires more code overall, but some of that code might be present in a contestant's comprehensive library. It uses an incremental convex hull, which is a data structure that maintains a convex hull of a set of points and allows us to efficiently (in logarithmic time) add points to the set while updating the convex hull if necessary.

The algorithm checks for a condition that we mentioned in the analysis for Part 1: each segment has the two endpoints of all other segments on different sides. The algorithm uses a rotating sweep line. Assume the endpoints of all input segments are swapped as necessary such that the segments point right (the first endpoint has an x coordinate no greater than the x coordinate of the second endpoint). Then, we sort the segments by slope and consider a rotating line that stops at all those slopes — that is, we iterate through the slopes in order. If we number the segments S_1, S_2, ..., $S_{\mathbf{N}}$ in that order, S_1 must have all left endpoints on one side, and all right endpoints on the other. S_2 is the same, except the left endpoint of S_1 goes with the right endpoints of all others, and vice versa. In general, for S_i we need to check for the left endpoint of all segments S_1, S_2, ..., S_{i-1} to be on one side together with the right endpoints of all segments S_{i+1}, S_{i+2}, ..., $S_{\mathbf{N}}$, and all other endpoints are on the other side. If we find an endpoint of S_j on the wrong side of S_i, then S_i and S_j do not intersect. If we find no such example, the answer is `MAGNIFICENT`.

If we knew the convex hull of all the points that are supposed to be on each side, we could use ternary search on the signed distance between the convex hull and the line to efficiently find the point from that set whose signed distance perpendicular to the current S_i is smallest (for the side where the distances are supposed to be positive) or largest (for the other side). If one of those finds us a point on the wrong side, we are done; otherwise, we know all other points are also on the correct side. Unfortunately, to keep those two convex hulls as we rotate would require us to both add and remove a point from each set. Removing is a lot harder to do, but we can avoid it.

When considering the slope of S_i, instead of using the convex hull of the full set on one side, we can use the convex hull of the left endpoints that are on that side, and separately, the convex hull of the right endpoints on that side. That leaves us one additional candidate to check for that side, but one of those is the optimal candidate. Since we are calculating left and right endpoints separately, the $4 \times \mathbf{N} - 1$ convex hulls we need are the ones of the set of left endpoints of the segments in a prefix of the list of segments, the left endpoints of the segments in a suffix of the list of segments, and similarly, the right endpoints of the segments in a suffix or prefix of the list of segments. We can calculate all of those convex hulls with a data structure that only provides addition of a point by calculating the convex hulls for prefixes in increasing order of segment index, and the ones for suffixes in decreasing order of segment index. Notice that this means we will calculate the convex hulls in an order different from the order in the original form of the algorithm.

We are doing O(**N**) insertions into the convex hull data structure and O(**N**) ternary searches, and each of these operations takes O(log **N**) time, making the time complexity of this algorithm also O(**N** log **N**).

For this particular use, we only need one half of the convex hull: the half that is closer to the line being inspected. In this half convex hull, the points in the hull are sorted by y coordinate, so a tree search can yield us the tentative insertion point, and we can maintain the convex hull by searching and inserting in a sorted tree. This is simple enough that it does not necessarily require prewritten code. Additionally, we can further simplify by using binary search on the angle between the convex hull and the line instead of the ternary search mentioned above.

Code Jam 2019 - World Finals

Analysis: Sorting Permutation Unit

A rotation-based solution

For simplicity, let's assume that none of the arrays we want to sort contain any repeated entries. If there are repeatred entries, we can arbitrarily choose a correct sorted order for them.

To illustrate the sorting algorithm, let's first ignore the constraint on the number of permutations, and use **N**-1 permutations: the i-th permutation ($1 \le i \le N-1$) swaps the i-th element with the **N**-th element.

For example, when **N** = 5, we use the following 4 permutations:

- 5 2 3 4 1
- 1 5 3 4 2
- 1 2 5 4 3
- 1 2 3 5 4

With this setup, we can use the **N**-th element as a "buffer" to sort the first **N**-1 elements. The algorithm is as follows:

1. If the element at position **N** is not the largest one, swap it to the correct position.
2. Otherwise, there are 2 cases:
    - The array is sorted, and we are done.
    - The array is not sorted, so we swap the **N**-th element with any element that is at an incorrect position (so that we can continue using it as buffer).
3. Repeat from step 1.

For example, let's consider the array [30, 50, 40, 10, 20]. We can:

- Swap 20 to the correct position: [30, 20, 40, 10, 50].
- Now 50 is at the last position, but the array is not sorted, so we swap it with any element that is at an incorrect position, for instance, 30: [50, 20, 40, 10, 30].
- Swap 30 to the correct position: [50, 20, 30, 10, 40].
- Swap 40 to the correct position: [50, 20, 30, 40, 10].
- Swap 10 to the correct position: [10, 20, 30, 40, 50].
- The array is now sorted.

Note that this solution uses **N**-1 permutations and 1.5**N** operations:

- **N** operations to swap **N** elements to their correct positions,
- At most **N**/2 to swap the **N**-th element in case it is the largest. This is because after we swap the largest element with an element at the wrong position, we won't need to do so again on the next step.

Reducing the number of permutations

To fit within the limit on the number of allowed permutations, we can instead use the following 5 operations:

- One permutation that swaps elements **N**-1 and **N** — the next-to-last and last elements.
- 4 permutations that rotate each of the elements from 1 to **N**-1 by 1, 3, 9 and 27, respectively. (Notice that element **N** remains the same.)

With these 4 permutations, when **N** ≤ 50, we can rotate the first **N**-1 elements by any amount (from 1 to **N**-2) in at most 6 operations. For example, to rotate by 26 = 9 + 9 + 3 + 3 + 1 + 1, we rotate by 9 two times, rotate by 3 two times, and then rotate by 1 two times. To rotate by 47, we use 27 + 9 + 9 + 1 + 1. (Equivalently, we can express any number less than 50 using a base three string with trinary digits summing to 6 or less. The smallest number that would take 7 or more is 53.)

Our new algorithm is similar to the one above.

1. If the element E at position **N** is not the largest one, swap it to the correct position as follows:
    - Apply rotations to the first **N**-1 elements until the (**N**-1)-th position is the slot where E should go.
    - Swap the (**N**-1)-th and **N**th positions, moving E into place (and some other element into the **N**-th position.)
2. Otherwise, E is the largest element. We need to temporarily move it out of the **N**-th position. We can swap in some element that is not already in its correct relative position among the first **N**-1 positions. We can use the same strategy as above to rotate to that element. If there is more than one element in the wrong relative position, we choose the element such that the amount we need to rotate to get it in place is minimized.
3. We repeat the above until all of the elements in the first **N**-1 positions are in the correct relative order. Then, we may need to do some final rotations to put them in the correct absolute positions.

For example, let's consider the array [30, 50, 40, 10, 20]. We will use 3 permutations:

- Swap the last 2 elements: 1 2 3 5 4
- Rotate the first **N**-1 elements by 1: 4 1 2 3 5
- Rotate the first **N**-1 elements by 3: 2 3 4 1 5

The algorithm works as follows:

- We first want to swap 20 to the relative position after 10:
    - Rotate the first **N**-1 elements by 3: [50, 40, 10, 30, 20]
    - Swap the last 2 elements: [50, 40, 10, 20, 30]
- Now we want to swap 30 to the relative position after 20:
    - Rotate the first **N**-1 elements by 3: [40, 10, 20, 50, 30]
    - Swap the last 2 elements: [40, 10, 20, 30, 50]
- The largest element, 50, is now at the last position. We need to swap it with some element at the wrong position. In this case, there is exactly one such element: 40.
    - Rotate the first **N**-1 elements by 3: [10, 20, 30, 40, 50]
    - Swap the last 2 elements: [10, 20, 30, 50, 40]
- Finally, we want to move 40 to its correct position:
    - Swap the last 2 elements: [10, 20, 30, 40, 50].

The number of operations we use are:

- 1.5**N** operations for swapping (as shown in our first algorithm)
- 6**N** operations for rotating the first **N**-1 elements, to swap them to correct relative positions.
- For the case where the largest element is at the **N**-th position, we need an addition of **N** operations for rotating. This is because we always rotate the minimum amount, after which the first step will rotate the array until it's back at the same relative position. So in total, the rotations in this step will rotate the array at most one full cycle.

Thus we are using total of 8.5**N** = 425 operations.

An even tighter solution

For each **N** ≤ 50, there also exists a set of 4 rotations that allows us to rotate the first **N**-1 elements by any amount (from 1 to **N**-2) in at most 4 operations, making use of the fact that rotating by **k** is the same as rotating by **k+N-1**. As an example, for **N** = 50, {1, 3, 12, 20} is one such set.

This yields a solution that uses just 325 operations in the worst case.

A randomized variant

In an alternative solution, instead of four rotations of the other= **N**-1 elements, we have four random involutions which swap randomly selected pairs of the **N**-1 elements.

Now we use the same algorithm as in the rotation solution, but instead of rotating elements into place, we use a sequence of the random permutations to move the right element to where it can be swapped with the buffer, and then apply the sequence in reverse to move everything back to where it was.

(We can use a breadth-first search to find the shortest sequence to move any given element into place.)

This process may result in a set of permutations that can't solve the input or can't solve it in few enough permutations, but we can simply retry by choosing a new set of random involutions, as getting a solution with under 450 permutations is highly likely.

Code Jam 2019 - World Finals

Analysis: Won't sum? Must now

The fact that we never need more than 3 palindromes is not obvious, and it was proved in a long, complicated paper. Numberphile made a video that explains the basics of the paper's argument. Knowing this upper bound is useful, but is not necessary to solve the problem.

Even if we haven't read every paper on arXiv for fun, we might also reasonably guess that the number of palindromes required is unlikely to be large because there are approximately sqrt(**S**) palindromes smaller than **S**, which means that there are approximately **S** palindrome pairs, which should result in Θ(**S**) numbers smaller than **S** that should be representable as sums of palindrome pairs. The number of triples is larger by another factor of sqrt(**S**), and their sums are all smaller than 3**S**, which does not leave much space to hide a number that could not be represented by a triple of palindromes, unless many triples add up to the same number.

The first step is to check if **S** is itself a palindrome. If it is a palindrome, then we are done. In the remaining explanation, we will assume that **S** is not a palindrome. Checking if a number is a palindrome takes O(log **S**) steps, one step per digit.

Test set 1

Key insight: There are only roughly 2×10^5 palindromic terms up to 10^{10}. In the analysis below, we will call this number P.

Sum of 2 palindromes?

To check if **S** is the sum of 2 palindromes, we simply loop through all palindromes, p, less than **S** and check if **S**-p is a palindrome. This takes O(P × log **S**) steps.

Sum of 3 palindromes!

We were told that every number is the sum of 3 palindromes, so we must find such a sum! On first glance, it seems that doing the same as above for 3 palindromes is too slow since there are $O(P^2)$ pairs of palindromes to check. However, for all numbers up to 10^{10}, one of the 3 palindromic terms is always at most 404 (which is needed for 1086412253, for example). Thus, as long as we search the pairs of palindromes in a way that searches triples containing a small value first, we will be fine.

Test set 2

Unfortunately, there are as many as 2×10^{20} palindromes less than 10^{40}, so it is hopeless to even iterate through the palindromes. Something quicker will be needed.

S = X + Y?

We will assume X ≥ Y. The first step is to fix the numbers of digits in X and Y. Note that the number of digits in X must be either (1) the number of digits in **S** or (2) one less than the number of digits in **S** (otherwise, X + Y < **S**).

Let's assume that X and Y have a different number of digits (we will deal with the same-number-of-digits case below). The trickiest part about this problem is dealing with the pesky carries that

occur while adding. For example, if we want to find 2 palindromes that add up to 2718281828 with lengths 10 and 8, then the first digit of X can be either 1 or 2 (depending whether there is a carry on the most significant digit). Because we don't know, let's try both! First, let's assume that we do not need a carry (so the first digit of X is 2). This also means that we can determine the least significant digit of Y by considering the sum modulo 10 (which also fills in the most significant digit of Y since it is a palindrome).

```
  ..........          2........2          2........2
+   ........   ->   +   ........   ->   +   6......6
============        ============        ============
  2718281828          2718281828          2718281828
```

At this point, to fill in the remaining unknown digits, we need a 9 digit palindrome and a 7 digit palindrome that sum to 2718281828 - (2000000002 + 60000006) = 658281820. This is a subproblem and we may recurse (with leading zeroes now allowed). The other option is that there is a carry on the most significant digit. In this case, the first digit of X must be 1.

When the numbers of digits in X and Y are different, there is always at least one unknown digit that we can determine. However, if X and Y have the same number of digits, this is not necessarily true. However, we can combine two simple observations to determine viable options for the first (and last) digits of both numbers. First, we consider the sums modulo 10. This narrows the possible values we may choose. Second, the number of digits in **S** compared to the number of digits in X (and Y) tells us if we want the sum of the most significant digits to be at least 10 (causing a carry). For example, if **S** has 5 digits, while X and Y only have 4 digits, then we must provide a carry in order for this sum to work. Similarly, if the number of digits in **S** and X are equal, then we cannot have a carry, so the sum must be at most 9. Even after these constraints, we still have some options (for example, if we need a sum of 8, we can use 0+8, 1+7, ... , 7+1, 8+0). Note that it doesn't matter* which of these options we take, since the only way they interact with other digits is with the carry.

* - There is one case where it matters: we must avoid creating a leading zero in a number.

Let D be the number of digits in **S**. There are O(D) possible lengths for X and Y. For each pair of lengths, we must decide if there is a carry on each digit on the left-hand side of **S**. Depending on implementation, this is $O(2^{D/2} \times D)$ operations. This leads to a total of $O(2^{D/2} \times D^2)$ operations. Note that instead of just computing one digit at a time, we can determine the whole "overhang" values. In the example above, we can determine that the first two digits are either 26 or 27:

```
  ..........          27......72          27......72
+   ........   ->   +   ........   ->   +   65....56
============        ============        ============
  2718281828          2718281828          2718281828
```

This means that we only need to make D/overhang choices for the carries instead of D/2 choices. Thus, when the overhang is large, those computations do not contribute heavily to the complexity. This reduces the total complexity to $O(2^{D/2} \times D)$ operations.

Sum of 3 palindromes

In the paper listed above, the authors describe an algorithm to write any number as the sum of 3 palindromes is described. This was obviously not needed to solve this problem, but it does require an algorithm to find the sum of three palindromes.

To solve this problem, we will use the fact that there are O(**S** sqrt(**S**)) triples of palindromes less than **S**, which means that each number can be written with an average of O(sqrt(**S**)) different triples. This allows us to randomly select palindromes as one of our three values, then use the

algorithm above to check whether the remaining value can be wrritten as the sum of 2 palindromes.

It is possible that there is a number that can only be represented in one different way as a sum of three palindromes, but we were unable to find any case that was not the sum of triples in many different ways. In fact, we were unable to find an input number such that one of the three numbers was more than 10801 (though, we expect that many such cases exist).